Rice's Theorem: Statement, Proof and Application to Antivirus Limits

Project 16 | Theory of Computation | Formal write-up (Member 1)

1. Preliminaries and notation

We fix a standard model of computation: deterministic Turing Machines (TMs). Programs and TMs are treated as interchangeable, since any program can be compiled to a TM and back.

- ⟨M⟩ denotes the encoding (description) of a TM M as a string.
- L(M) is the language recognised by M: the set of inputs on which M enters its accepting state.
- A language is *Turing-recognisable* (RE) if some TM recognises it. It is *decidable* if some TM that halts on every input accepts exactly its members.
- A_TM = { ⟨M, w⟩ : M accepts w }.

Fact 0 (Turing, 1936). A_TM is undecidable. We use this as the starting point for every proof below.

2. Definitions

Definition 1 (Semantic property). A property of TMs is a set P of TM descriptions. P is *semantic* if membership depends only on the language recognised: whenever $L(M_1) = L(M_2)$, we have $\langle M_1 \rangle \in P$ if and only if $\langle M_2 \rangle \in P$.

Equivalently, a semantic property is determined by a set S of RE languages: P_S = { ⟨M⟩ : L(M) ∈ S }.

Definition 2 (Non-trivial). A semantic property P_S is *non-trivial* if S is neither empty nor the set of all RE languages. In other words, at least one TM has the property and at least one TM does not.

A property that depends on the *code* rather than the behaviour (for example, "M has at least 10 states") is *syntactic*, not semantic, and Rice's Theorem does not apply to it.

3. Statement

Theorem (Rice, 1953). Let S be a non-trivial set of RE languages. Then the language $P_S = \{ \langle M \rangle : L(M) \in S \}$ is undecidable.

In plain words: no algorithm can decide, for every program, any non-trivial question about what the program does.

4. Proof

We reduce A_{TM} to P_S. Suppose, for contradiction, that a TM R decides P_S.

Step 1: normalise so that $\emptyset \notin S$. Suppose $\emptyset \in S$. Let $\bar{S}$ be the set of all RE languages not in S. Then $\bar{S}$ is also non-trivial and $\emptyset \notin \bar{S}$. A decider for $P_{\bar{S}}$ is obtained from R by swapping its accept and reject answers, and vice versa. So it is enough to prove the theorem when $\emptyset \notin S$, and we assume this from now on.

Step 2: pick a witness. Because S is non-trivial, it is non-empty. Choose a language $A \in S$. Since $\emptyset \notin S$, the language A is non-empty. Fix a TM M_A with $L(M_A) = A$.

Step 3: build the reduction. Define a TM D that, on input $\langle M, w \rangle$, does the following.

1. Construct the description of a new TM M' that behaves as follows on an input x:
   - (a) Simulate M on w. If M does not accept w, M' rejects x or runs forever (either choice works).
   - (b) If M accepts w, run M_A on x and accept x exactly when M_A accepts.
2. Run R on $\langle M' \rangle$. If R accepts, D accepts. If R rejects, D rejects.

The map $\langle M, w \rangle \mapsto \langle M' \rangle$ is computable: M' is obtained by hard-coding M, w and M_A into a fixed template, so step 1 always halts.

Step 4: correctness. There are two cases.

- *M accepts w.* Step (a) always succeeds, so M' behaves exactly like M_A on every input, and $L(M') = A \in S$. Hence R accepts $\langle M' \rangle$, and D accepts.
- *M does not accept w.* Step (b) is never reached, so M' accepts nothing, and $L(M') = \emptyset \notin S$. Hence R rejects $\langle M' \rangle$, and D rejects.

Therefore D accepts $\langle M, w \rangle$ if and only if M accepts w, and D halts on every input (R is a decider and step 1 halts). So D decides A_{TM}, contradicting Fact 0. Hence R cannot exist, and P_S is undecidable. ∎

Corollaries (all undecidable). "Is L(M) empty?", "Is L(M) regular?", "Does M accept at least five strings?", and "Do M_1 and M_2 recognise the same language?" (the last by a related argument).

5. Application to antivirus

Modelling malware. Rice's Theorem speaks about recognised languages, but malware is about actions. We therefore model a malicious action as reaching a distinguished state q_mal, and define

MAL = { ⟨M⟩ : there exists an input x on which M reaches q_mal }.

MAL is semantic, because it depends only on the behaviour of M. It is non-trivial, because some machines reach q_mal and some never do.

Theorem (no perfect antivirus). MAL is undecidable. Hence no algorithm perfect_av can classify every program correctly.

Proof (direct reduction from A_TM). Given ⟨M, w⟩, build a program P that ignores its input, simulates M on w, and enters q_mal only if M accepts w. Then ⟨P⟩ ∈ MAL if and only if M accepts w. A decider for MAL would therefore decide A_TM, which is impossible. ∎

The paradox (diagonal) argument. Suppose perfect_av decides MAL. By Kleene's Recursion Theorem there is a program D that can obtain its own description ⟨D⟩ and then does the following: if perfect_av(⟨D⟩) says "malware", D halts without ever entering q_mal; otherwise D enters q_mal. In both cases perfect_av is wrong about D, a contradiction. The Python demo (the paradox_program function) realises this self-reference by name, and the Recursion Theorem is the formal justification that such self-reference is always possible.

6. Remarks and limits of the result

1. **Recognisable but not decidable.** MAL is Turing-recognisable: run every program on every input in an interleaved (dovetailed) way and accept as soon as q_mal is reached. So a detector can eventually confirm malware that shows itself, but can never certify a program as safe. The complement of MAL is not recognisable, since otherwise MAL would be decidable.
2. **Restricted settings are different.** The theorem is about arbitrary programs. For restricted classes (for example, programs with no loops or with bounded memory), such properties can be decidable.

3. **Heuristics are approximations.** Real tools (signature, behavioural and ML-based detection) give up exactness. Every such method has false positives, false negatives, or both, which is what Part 5 of our script illustrates.
4. **Idealisation.** Real detectors also use non-semantic signals such as code patterns and reputation. These improve practical accuracy but cannot remove the theoretical limit on semantic classification.

7. Link to our implementation

Theory	Python demo
Turing Machine, A_TM, halting	`TuringMachine` class, `run` with an optional step limit
Program P built from ⟨M, w⟩	`make_program` and `bounded_av` (Part 2/3)
Failure of any fixed budget	Budgets 100 and 1,000 miss the machine that halts after 50,000 steps
Paradox program	`make_paradox` (Part 4)
Heuristic detection with errors	Signature, behaviour and score engines (Part 5)

References

1. M. Sipser, *Introduction to the Theory of Computation*, Cengage (undecidability chapter; Rice's Theorem appears as a problem).
2. A. M. Turing, "On Computable Numbers, with an Application to the Entscheidungsproblem", Proc. London Math. Soc., 1936.
3. H. G. Rice, "Classes of Recursively Enumerable Sets and Their Decision Problems", Trans. AMS, 1953.
4. S. C. Kleene, "On Notation for Ordinal Numbers", J. Symbolic Logic, 1938 (Recursion Theorem).